

# GROWTH FUNCTION

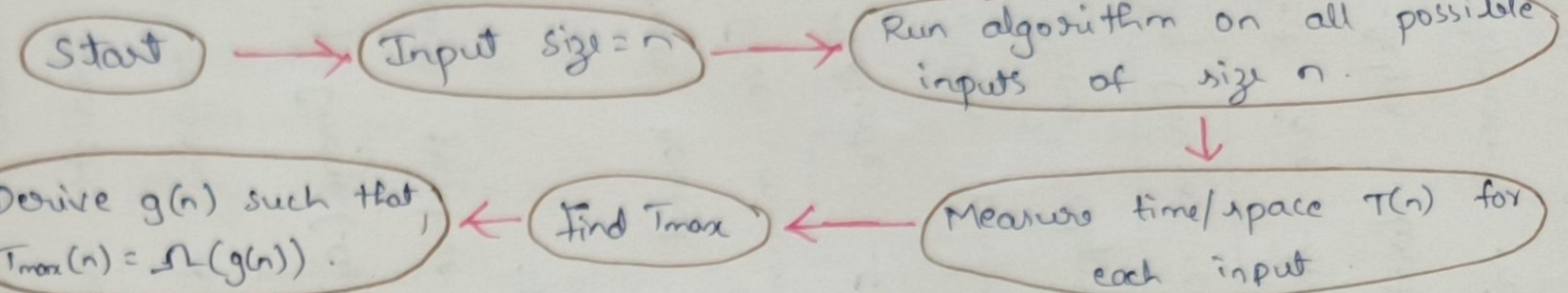
## ALGORITHM

A finite sequence of well defined instructions to solve a problem.



## GROWTH FUNCTION

A function that describes how the resource usage (time/space) of an algorithm with input size  $n$ .



## Advantages -

- \* Shows minimum execution time
- \* Helps compare algorithms.
- \* Predicts performance
- \* Useful in optimization
- \* Provides theoretic lower bound.

## Big $\Omega(n)$

provides a lower bound on the growth of a function.

$$\Omega(g(n)) = \{f(n) : \exists c > 0, n_0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n), \forall n \geq n_0\}$$

## LOWER BOUND

Best possible performance any algorithm can achieve for a problem.

## BACK CASE (WORST CASE)

The input that causes an algorithm to take the maximum time/space.

## EXAMPLE - Linear Search

Time Complexity  $T(n) = cn$

Growth Function  $g(n) = n$ .

Big- $\Omega$  (lower Bound):  $T(n) = \Omega(n)$

At least linear time is required.

Worst Case: Element is at last position or not present.



## Algorithm Characteristics -

- well defined inputs & outputs
- Finite No. of steps
- Unambiguous instructions.
- Terminates after execution.

## Growth Function Importance -

- Measures algorithm efficiency
- Predicts scalability
- Compares different algorithms.

Big Omega ( $\Omega$ ) - An algorithm belongs to  $\Omega(g(n))$  if there exist positive constants  $c$  &  $n_0$  such that,

$$T(n) \geq c \times g(n) \quad \text{for all } n \geq n_0$$

## Lower bound Importance -

- Gives minimum performance guarantee.
- Indicates fastest possible execution.
- Helps compare algorithms.
- Prove algorithm efficiency.

Big Omega notation mathematically represents lower bound

## Best Case Analysis Importance -

- Helps understand optimal performance.
- Represents minimum Resource usage.
- Shows fastest Execution.



2.

## Recurrence Relation

- $T(n)$  in terms of  $T$ .
- An eq<sup>n</sup> that defines a  $f(n)$  in terms of its values in smaller inputs.

## Tips -

- Choose  $f(n)$  close to the non-recursive part.
- Always prove  $n \geq n_0$
- Use mathematic induction.

## Substitution method

Technique to solve recurrences by guessing a bound and proving it by mathematical Induction.



## STEPS

- \* Choose a guess for  $T(n)$
- \* Assume  $T(k) \leq c \cdot f(k)$  for all  $k < n$ .
- \* Substitute in  $T(n)$
- \* Simplify & compare  $c \cdot f(n)$
- \* Choose constant  $c$  &  $n_0$  to satisfy inequality.
- \* Conclude by induction.

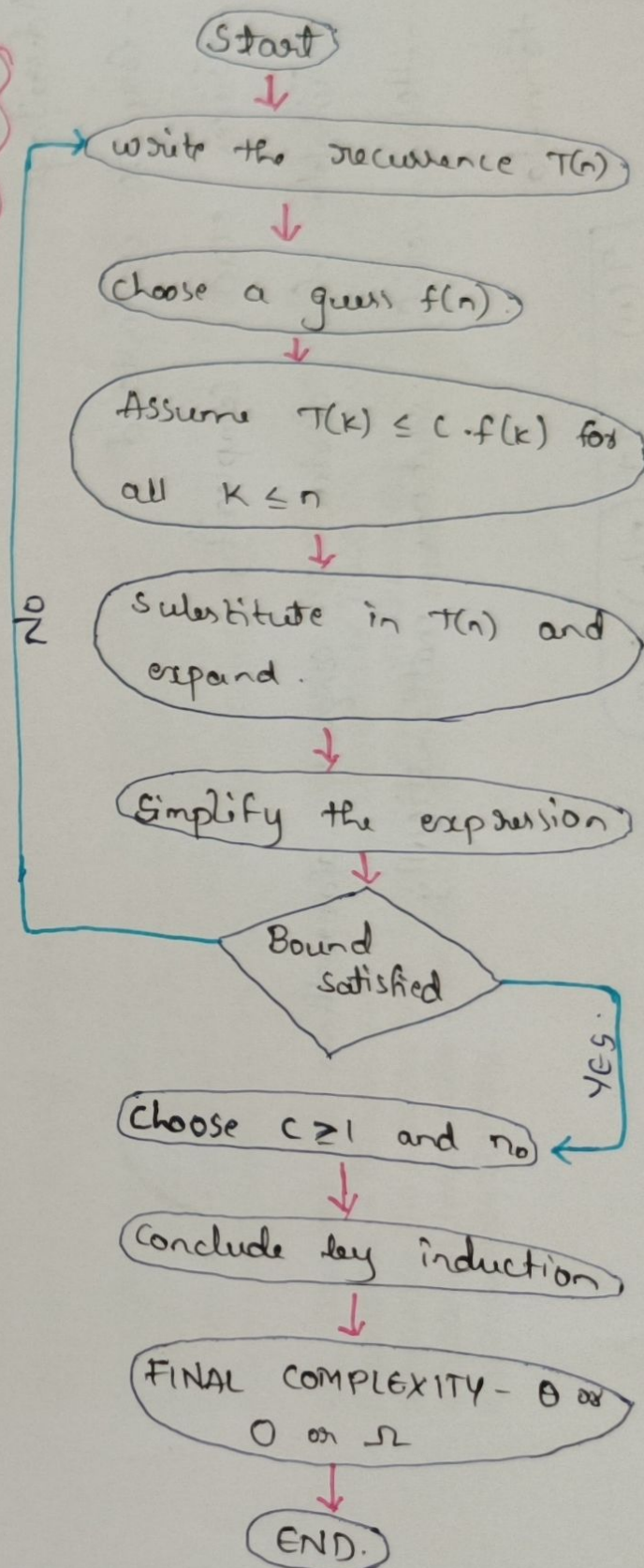
## Expression -

The guessed form of sol<sup>n</sup> (using  $\Theta, \Omega, O$  notation).

Simplification - Algebraic steps to reduce recurrence and expression to simple form.

## Final Complexity

The asymptotic bound ( $\Theta, O, \Omega$ ) that describes growth of the function.





Why is Recurrence relation used?

- To analyse time complexity of recursive algorithms.
- To solve divide & conquer algorithms.
- To derive final asymptotic complexity using methods like substitution, Recursion Tree or Master Theorem.

Flow -

Recursive Algorithm



Break into smaller problems



write Recurrence relation



Solve Recurrence



Find time complexity



Analyse algorithm efficiency.

Advantages -

- easy to understand
- Gives exact complexity
- Suitable for divide & conquer algorithms.
- Helps verify solutions mathematically.

Formula -

$$T(n) = 2T(n/2) + n$$

$$T(n) = \Theta(n \log n) \quad \text{— verify using Induction.}$$